

Glossary - Cross-Domain Circuit Analysis

Glossary — Cross-Domain Circuit Analysis

Purpose: Plain-language definitions of key concepts used in the paper, for explaining the work to others.

Companion to: CROSS_DOMAIN_CIRCUIT_PAPER.md

Table of Contents

- Jaccard Similarity
- Activation Energy
- Bottleneck Tax
- Cosine Similarity on Energy Profiles
- Is Energy Finite? (Addressing the Metaphor)
- Steering Experiments: How Features Were Selected
- Direct Logit Attribution: How the Output Token is Chosen

Jaccard Similarity

Quick definition: A number between 0 and 1 that measures how much two sets overlap.

Formula: $J(A, B) = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{size of overlap}}{\text{size of combined set}}$

How to read the values:

- **J = 0** → the two sets share nothing
- **J = 1** → the two sets are identical
- **J = 0.5** → half the elements are shared

Worked example:

Set A = {apple, orange, banana, grape} Set B = {apple, orange, pear, kiwi}

- Intersection: {apple, orange} → size 2
- Union: {apple, orange, banana, grape, pear, kiwi} → size 6
- **J = 2/6 ≈ 0.33**

How we use it: Every circuit has a set of SAE features that activate during the prompt. We compute Jaccard over those feature sets to ask "how similar are these two circuits?"

Example results from the paper:

Comparison	Jaccard	Interpretation
Gold vs Sodium (same template, different facts)	0.60	Circuits share 60% of features

Comparison	Jaccard	Interpretation
Gold template-A vs Gold template-B (same fact, different templates)	0.30	Only 30% shared
Chemistry vs Geography (different domain)	0.11	Very different circuits
GEMMA Geography prompts (within-domain)	0.40	Highest convergence within-domain
GEMMA History prompts (within-domain)	0.11	Lowest convergence within-domain

Why we chose it:

- Simple and interpretable
- Doesn't care about set size — two small sets with perfect overlap still score 1.0
- Ignores element ordering and magnitude

Limitation: Treats all features as equally important. A strongly-activated feature counts the same as one that barely fires.

Activation Energy

Quick definition: A measure of how much "work" a neural network layer is doing, quantified by the magnitudes of feature activations.

How we compute it:

- **Node energy** = sum of activation magnitudes across all SAE features in that layer
- **Total activation energy** = sum of node energies across the entire circuit
- **Layer energy fraction** = (energy at layer L) / (total circuit energy) — what % of the circuit's work happens at layer L

Why call it "energy"? Physics analogy. High energy = many features strongly firing. Low energy = sparse activation. The name captures the intuition that a layer with high activation magnitudes is "doing more work."

How it looks in the paper:

Energy profiles plot layer-energy-fraction against layer depth for each circuit. We found two distinct patterns:

- **GEMMA front-loads energy:** 54% of total energy is in layers 0-12 (first half). Cumulative 50% reached by L11.
- **QWEN back-loads energy:** Only 31% of energy in the first half. Cumulative 50% not reached until L28.

These are fundamentally different computational strategies, but both produce comparable factual recall.

Key property: Energy profiles are **architecture-determined, not domain-determined**.

- Within-model cosine similarity (across chemistry/geography/history): **0.978**
- Between-model cosine similarity: **0.696**
- Architecture explains roughly 14× more variance than knowledge domain

Why this matters: Energy profiles are robust to prompt-template confounds (see Section 5.9). Feature IDs change when you rephrase prompts, but *where* computation happens across layers stays architecturally locked. That's why the architecture-dominance claim survives the format-variation critique.

Bottleneck Tax

Quick definition: The empirical finding that *higher activation magnitude at a bottleneck layer correlates with worse model confidence*.* The name is a memorable label; the theoretical grounding is information bottleneck theory (Tishby & Zaslavsky, 2015), which predicts that over-compression at an intermediate layer limits the information downstream layers can use for prediction.

Important note on framing: This is a **correlation with a theoretical interpretation, not a proven causal mechanism**. "Energy" here means summed feature activation magnitudes, NOT a conserved physical quantity (see the "Is Energy Finite?" entry). Early drafts used "energy budget" language; the final paper grounds the finding in information bottleneck theory instead.

The core correlations (GEMMA-2-2B):

Layer	Role	Correlation with confidence	Direction
L6	Primary bottleneck	$r = -0.684, p < 0.0001$	Negative (tax)
L10	Mid-layer	$r = -0.601$	Negative
L1	Input	$r = +0.634$	Positive
L13	Post-bottleneck	$r = +0.601$	Positive
L16	Post-bottleneck	$r = +0.598$	Positive

More energy at L6 → worse predictions. More energy at L13/L16 → better predictions.

The intuition (doorway analogy — use cautiously): Think of a narrow doorway in a hallway.

- Information has to squeeze through the doorway (bottleneck at L6)
- The *useful assembly* happens in the rooms after the doorway (L13, L16)
- When too much is compressed at the doorway, less useful information reaches the downstream rooms for assembly

This is a metaphor to build intuition. The **formal version** comes from information bottleneck theory: the mutual information $I(X; T_L)$ between input X and the representation at layer L bounds what downstream layers can recover. Higher compression at the bottleneck layer reduces this mutual information, which limits downstream evidence accumulation.

Note: The bottleneck both filters information (potentially helpful, by discarding noise) AND limits downstream information (potentially harmful, if it discards signal). The paper's finding is that on average the harm dominates — circuits with heavier bottleneck activation tend to produce less confident predictions.

What we ruled out:

- **Bottleneck convergence** (how sharply paths funnel): $r = 0.27$, NOT significant
- **Bottleneck energy fraction** (what % of energy is at bottleneck): $r = 0.26$, NOT significant
- **Total activation energy**: $r = 0.53$, Bonferroni-significant

So it's not about having a "sharper" or "cleaner" bottleneck — it's about not *over-spending* energy there.

Why this matters: This is one of the paper's most novel contributions because it **connects circuit structure directly to behavior**. Earlier work (Meng et al. 2022) showed specific layers matter for factual recall via causal intervention. Our bottleneck tax is the correlational version: just by measuring *where* energy lives in a circuit, you can predict *how confident* the model will be.

QWEN3-4B contrast: QWEN shows zero Bonferroni-significant layer-energy-confidence correlations. Its late-bottleneck architecture (L22-25) diffuses the confidence signal across many layers instead of localizing it. The bottleneck tax appears specific to early-bottleneck architectures like GEMMA.

One-line summary: Bottleneck tax = an empirical correlation (higher L6 activation → lower confidence) grounded in information bottleneck theory (Tishby 2015), not a literal energy-budget mechanism.

How to explain it to someone: > "We found that the more a circuit's activation is concentrated at its bottleneck layer (L6 in GEMMA), the less confident its prediction tends to be — $r = -0.684$, Bonferroni-significant. This is consistent with information bottleneck theory, which predicts that heavy compression at intermediate layers limits the information downstream layers can use. We call the pattern the 'bottleneck tax' for memorability, but it's a correlational finding with a theoretical interpretation, not a proven causal mechanism."

Cosine Similarity on Energy Profiles

Quick definition: A number between 0 and 1 measuring whether two circuits have the same *shape* of layer-by-layer energy distribution, regardless of total magnitude.

What the inputs are: Each circuit produces a vector of N numbers (26 for GEMMA, 36 for QWEN) — the proportion of total circuit energy at each layer. Example:

GEMMA circuit → [0.08, 0.12, 0.15, 0.09, 0.06, 0.04, 0.03, 0.02, ...]

These sum to 1.0 (they're fractions).

Why cosine specifically:

1. **Shape over magnitude.** Measures the *angle* between vectors, not their length. Two circuits concentrating energy at L6 will score as similar even if one has much more absolute energy. We care about *where* computation happens, not how much.
2. **Standard in ML.** Used ubiquitously for comparing embeddings, feature vectors, and activation patterns.
3. **Interpretable.** 1 = identical shape, 0 = completely different shape.

Alternatives we didn't use and why:

- **Euclidean distance:** dominated by absolute magnitude differences, not shape
- **KL divergence:** asymmetric, treats vectors as probability distributions (which is OK since ours sum to 1, but harder to interpret as a "similarity")
- **Correlation:** adds a mean-centering step that's harder to interpret here

Precedents in the literature:

- **Representational Similarity Analysis (RSA)** — Kriegeskorte, Mur, & Bandettini (2008), *Frontiers in Systems Neuroscience*. Standard for comparing activation patterns across brains/models/conditions.
- **Centered Kernel Alignment (CKA)** — Kornblith, Norouzi, Lee, & Hinton (2019), *ICML*, "Similarity of Neural Network Representations Revisited." Modern method for comparing activations across networks.
- **Cosine on SAE features** — Bricken et al. (2023), Templeton et al. (2024) use cosine similarity on feature activations.

What's novel in our application: Applying cosine similarity specifically to **per-layer energy fractions aggregated across a circuit**, rather than comparing individual activation vectors. It's a reasonable extension of standard practice.

The key results (Section 5.5.1):

- Within-model cosine similarity: **0.978**
- Between-model cosine similarity: **0.696** (after depth normalization)
- Mann-Whitney U test: **$p < 0.000001$** (distributions don't overlap)
- Bootstrap 95% CI on within-model mean: **[0.975, 0.981]**

Where we're vulnerable to reviewer critique:

- We didn't formally benchmark cosine against KL divergence or Earth Mover's Distance. A reviewer could reasonably ask "would the pattern hold with a different metric?"
- The "~14× more variance" claim divides the between-model gap (0.282) by the within-model domain gap (~0.02). This is a rough comparison, not a formal variance decomposition.

How to explain it: > "Cosine similarity measures whether two circuits concentrate their computation at the same layers — we care about the shape of where work happens, not the absolute amount. This extends representational similarity analysis (Kriegeskorte 2008) and modern activation-comparison methods like CKA (Kornblith 2019). Our finding that within-model scores cluster at 0.978 while between-model scores drop to 0.696 is statistically robust (Mann-Whitney $p < 0.000001$)."

Is Energy Finite? (Addressing the Metaphor)

Quick definition: Our paper frames the bottleneck tax as if the model has a "finite energy budget" that must be spent somewhere. **This is a metaphor, not a mechanism.** Neural networks have no conservation law for activation energy.

The physical reality:

- Each layer's activations are the output of learned linear + nonlinear transformations applied independently
- Layer 6 can activate 0 features or 16,000 features without "using up" anything Layer 13 needs
- Activation magnitudes are bounded by input magnitudes and learned weights, not by a global pool

What IS finite in a neural network:

- **Parameters** (fixed capacity)
- **Compute per forward pass** (fixed FLOPs)
- **Input information content** (bounded by prompt)

But these are different from a "shared energy budget."

So what does the bottleneck tax actually capture?

The correlation ($r = -0.684$ between L6 energy fraction and confidence) is **real data**. The *interpretation* as a resource tradeoff is where we're on thinner ground. Three more rigorous ways to frame the same finding:

Option 1: Correlational (safest for publication)

"Layer 6 energy fraction negatively correlates with output confidence across 30 GEMMA circuits. We do not claim a direct causal mechanism; this pattern could reflect upstream prompt difficulty, domain differences, or genuine compression-accumulation tradeoffs."

Option 2: Information-theoretic (stronger, with citation)

Following **Tishby & Zaslavsky (2015)**, "Deep Learning and the Information Bottleneck Principle" (ITW) and **Shwartz-Ziv & Tishby (2017)**, "Opening the Black Box of Deep Neural Networks via Information":

> "Information bottleneck theory predicts that heavy compression at an intermediate layer limits the information available to downstream layers for reconstruction and prediction. The negative correlation between L6 energy fraction and confidence ($r = -0.684$) is consistent with this prediction: circuits that concentrate more processing at the bottleneck may be compressing harder, leaving less information for evidence accumulation in later layers."

This framing uses **information** (which does have a precise mathematical meaning in this context) rather than **energy** (which doesn't, in this context).

Option 3: Economic metaphor (what we currently wrote)

"Heavy processing at the bottleneck consumes resources that could otherwise contribute to confidence." Evocative but not mechanistically grounded.

References that do NOT support "energy is finite":

- **Energy-based models** (LeCun, Hinton): use "energy" as a scalar objective function over configurations — completely different concept
- **Activation atlases** (Olah et al., Distill): use activation magnitudes as features without framing them as finite
- **Hardware energy consumption** (MLPerf, EfficientNet literature): about Joules used by chips, not about activations

Recommendation for publication:

We should soften Section 6.4 to either Option 1 (correlational) or Option 2 (information-theoretic, with citation). The current "energy tax" framing is evocative but a reviewer who pushes on the mechanism will find nothing solid behind it.

How to explain it honestly: > "The paper uses 'energy' as a convenient shorthand for activation magnitude, not as a claim that the model has a literal conserved quantity. The bottleneck tax is a correlation, not a mechanism. The best theoretical grounding comes from information bottleneck theory (Tishby & Zaslavsky 2015), which predicts that compression at intermediate layers reduces information available downstream — consistent with our observation that high L6 energy predicts low confidence."

What this means for the paper:

- The **data** ($r = -0.684$, Bonferroni-significant) stands.
- The **finding** (certain layers' energy predicts confidence) stands.
- The **interpretation** (why) should be softened to information-theoretic language or pure correlational language.
- The name "bottleneck tax" is fine as a memorable label, but the framing around it should acknowledge it's a metaphor.

Steering Experiments: How Features Were Selected

Quick definition: Steering is a causal validation technique where we clamp a single SAE feature's activation to a fixed value (positive = amplify, negative = suppress) and measure whether the model's output changes. We ran 80 such experiments across three batches, each using a different criterion to choose which features to steer.

The basic mechanism: For each experiment, we made one Neuronpedia API call with one feature, one prompt, and one strength value:

```

POST /api/steer
prompt = "The Titanic sank in"
feature = L7_F4828270 (one feature at a time, never combined)
strength = +20 (or -20;  $\pm 20$  was used in every experiment)

```

The API returned both the unsteered (DEFAULT) and steered (STEERED) outputs along with token-level logprobs. We measured: (a) text change (binary), (b) KL divergence between baseline and steered token distributions, (c) logprob shift in the top token.

Three batches, three different selection criteria:

Batch	Criterion	Source pool	Features	Experiments
D4	Cross-circuit frequency, ranks 1-5	Stage 1.5 bottleneck library (244 features that appeared as bottlenecks across the 60-circuit dataset)	L0_F1813559, L3_F5150441, L4_F110446948, L6_F2586668, L24_F88478228	20
D5	Cross-circuit frequency, ranks 6-10 (extending D4)	Same library	L1_F99962728, L2_F25751073, L5_F7993995, L7_F4828270, L9_F125286525	30
D6	Essential-pathway membership (topology)	1,000 features extracted from minimal-pathway analysis on 30 GEMMA circuits	L0_F64712375, L1_F1736314, L21_F5479683, L24_F18002975, L25_F50014975	30

Cross-circuit frequency = the number of distinct circuits (out of 60) in which a feature appears as a bottleneck (convergence $\geq 60\%$ in the traceback analysis). High frequency = "this feature shows up everywhere."

Essential-pathway membership = the number of distinct circuits whose minimum-viable input-to-output pathway includes this feature. High pathway membership = "this feature is on the critical route, not the redundant scaffolding." Only $\sim 6\%$ of nodes per circuit are on the essential pathway.

Why three batches:

D4 was the initial validation. D5 expanded it because only 50% of D4 experiments produced text changes and we wanted more statistical power. D6 was added when an unexpected finding emerged from D5: the most-frequent feature (L2_F25751073, 11 circuits) produced **zero** text changes, while a less-frequent one (L7_F4828270, 9 circuits) was the most causally effective. This dissociation between frequency and causation motivated testing a different criterion (topology) to see whether essential-pathway features would do better.

Pre-experiment cross-check: Of the 10 frequency-selected features in D4+D5, only 5 were on essential pathways (L0_F1813559, L1_F99962728, L2_F25751073, L3_F5150441, L24_F88478228). The other 5 were frequent but NOT essential — they appear often but aren't on critical paths. This split made D6 a clean comparison.

What was NOT varied:

- **Strength.** All 80 experiments used ± 20 only. No dose-response curve. (See "Why ± 20 ?" below.)
- **Number of features per intervention.** Always one feature at a time. No compound steering.
- **Random seed.** All experiments used seed=42, temperature=0 for reproducibility.

Why ±20 strength specifically?

The Neuronpedia steering API accepts strengths from approximately -50 to +50. We used ±20 across all 80 experiments. Three reasons:

- 1. **It avoids two empirical failure modes.** At ±5 or ±10, most features produce no visible text change — too weak to distinguish causally-inert features from under-stimulated ones. At ±50 or higher, the model often produces nonsensical output ("activation overflow"), with effects that are real but uninterpretable. ±20 is in the middle: strong enough to detect causal influence, weak enough that surviving outputs remain grammatical.
- 2. **It is a community-convention default.** Templeton et al. (2024) used clamping values roughly 5-10× a feature's natural maximum for the "Golden Gate Bridge" demonstration. Turner et al. (2023, activation engineering) used coefficients typically in the 1-15× range. Neuronpedia's documentation suggests ±10 to ±30 as illustrative defaults. ±20 sits in the middle of this established range. Reporting ±20 also keeps our results directly comparable to other groups using the same convention.
- 3. **Practical: API rate-limit budget.** Each steering call costs ~36 seconds at Neuronpedia's 100-call/hour limit. Our total budget was 80 calls. We spent it on breadth (15 features × 3-6 prompts) rather than depth (a full dose-response curve at ±5/±10/±20/±50). Building the dose-response curve is listed as Future Work.

Empirical validation that ±20 was a reasonable choice:

Batch	Text change rate at ±20	Interpretation
D4	50% (10/20)	Some features change outputs, others don't — clean separation
D5	23% (7/30)	Most features ineffective at this strength — selection matters
D6	27% (8/30)	Comparable to D5 — different selection, similar overall rate

These rates fall in the useful 20-50% band: not so low that everything looks inert, not so high that everything looks effective. Both ends of the spectrum would have hidden the three-tier dissociation finding by collapsing it into "all features causal" or "no features causal."

Limitations of single-strength testing:

We do not know whether the three-tier dissociation persists at other strengths. It is possible that at ±50, essential-pathway features dominate text-change rates as well as KL divergence (because circuit redundancy can no longer absorb the larger perturbation). It is also possible that at ±5, only frequency-on-pathway features show any effect. The single-point sample at ±20 captures one slice of an unmeasured curve.

The headline finding (the "three-tier dissociation"):

Group	Selection method	Text change rate	Mean KL divergence
Essential-pathway features (D6)	Topology	26.7% (8/30)	1.448
D5 features that happen to be ON pathway	Frequency-on-pathway	18.8% (3/16)	~0.4
D5 features that are OFF pathway	Frequency-off-pathway	33.3% (6/18)	~0.4

Essential-pathway features produced the strongest distributional perturbations (highest KL) but circuit redundancy absorbed most of the perturbation before the output layer. Output determinism by domain (chemistry 0%,

geography 20%, history 60%) governed text-level susceptibility regardless of which features were steered.

One-line summary: We steered 15 unique features (5 per batch × 3 batches) at ±20 strength on 3-6 prompts each, with each batch chosen by a different selection criterion (frequency × 2, then topology) — and the three-tier dissociation finding emerged from comparing the three batches against each other.

How to explain it to someone: > "We did 80 single-feature steering experiments split into three batches. The first two batches selected features by how often they appeared across our 60-circuit dataset; the third selected by whether they sit on the essential information pathway. Each feature was steered at ±20 strength on three target prompts (one per knowledge domain). The point of using three different selection criteria was to test whether structural importance metrics — frequency vs topology — actually predict causal influence. Neither does very well, which is itself a finding."

Direct Logit Attribution: How the Output Token is Chosen

Quick definition: The mechanism by which activations of output-layer SAE features get translated into a chosen vocabulary token. Each feature has a "decoder direction" in residual-stream space; that direction projected through the unembedding matrix tells you which tokens the feature pushes toward and which it pushes away from. The model selects whichever token has the highest resulting logit.

The full path from output features to chosen token:

```
Output-layer SAE features (L25 in Gemma, L35 in Qwen)
↓
Each feature has a decoder direction d_f (a vector in residual-stream space)
↓
Sum of (activation × d_f) reconstructs the residual stream
↓
Final LayerNorm
↓
Multiply by unembedding matrix W_U (vocab_size × hidden_dim)
↓
One logit per vocabulary token
↓
argmax (or softmax sample at temperature > 0) → the chosen token
```

The math behind individual feature contributions:

For a single feature f with decoder direction d_f , activation magnitude a_f , and unembedding column $W_U[:, t]$ for token t :

$$\text{logit_contribution}(f, t) = a_f \times (d_f \cdot W_U[:, t])$$

This dot product is essentially the alignment between the feature's decoder direction and the token's unembedding row. **High positive value** = "this feature pushes toward this token." **High negative value** = "this feature suppresses this token." **Near zero** = "this feature has no direct effect on this token."

This calculation is called **direct logit attribution (DLA)**, sometimes also called "logit lens applied to SAE features."

Two kinds of "output node" in our pipeline:

1. **Logit nodes** — these represent specific vocabulary tokens. The Top-K (K=5) highest-logit tokens become the anchor nodes that traceback graphing starts from. For "The capital of Japan is", the top logit nodes might be "Tokyo", "the", "a", "called", "known".
2. **Final-layer SAE feature nodes** — SAE features at the last layer (L25 for Gemma, L35 for Qwen) whose decoders project toward those top-K tokens. Their connections to logit nodes in the attribution graph are weighted by direct logit attribution.

Why output-feature convergence is meaningful (the §5.4 finding):

Same-domain circuits share **67% of their output features** in Qwen history (vs only 15% of path features). The mechanism: different history prompts ("WWII ended in", "Columbus reached the Americas in") need to produce different tokens (1945, 1492), but those tokens share decoder structure — both 4-digit years, both common in encyclopedic text. The same output features push toward the year-token region of vocabulary space. What differs across prompts is how earlier layers route information to those shared output features.

This is the formal basis for "convergent outputs, divergent paths."

The selection step (at temperature=0):

`argmax(logits)`. That's it. No beam search, no sampling. The token with the highest logit wins. This is why steering can flip predictions: shifting one feature's activation ripples through the decoder math, changes some logits, and can flip which token wins the `argmax`.

Why the bottleneck tax connects to this:

If energy gets concentrated at L6 (compression bottleneck), less informative residual-stream content reaches L25 where the unembedding actually consumes it. The output features at L25 still fire — but on degraded input — producing a flatter logit distribution (lower confidence). The information bottleneck framing captures this: $I(X; T_{L25})$ is bounded by $I(X; T_{L6})$, and a flatter logit distribution means less peaked `argmax` probability.

Tools that compute DLA for you:

- **Neuronpedia feature dashboards** show top positive and negative logits per feature in their `top_logits` and `bottom_logits` fields
- **TransformerLens** has `apply_ln_to_stack()` and unembedding helpers for projecting activations through the final layers manually
- The Neuronpedia feature explanation API returns these fields when available

One-line summary: Each output-layer feature has a decoder direction that, when projected through the model's unembedding matrix, produces a contribution to every vocabulary token's logit. The chosen token is just `argmax` of the sum of these contributions across all active features.

How to explain it to someone: > "The model doesn't 'pick' tokens directly from features. Each output-layer feature has a learned vector that encodes which tokens it pushes toward when active. Those vectors get projected through the unembedding matrix to produce logits, and the highest-logit token wins. Two features that look completely different in semantic terms can both push toward 'Tokyo' if their decoder vectors happen to align with the unembedding row for 'Tokyo' — which is why we see same-domain prompts converging on shared output features even when their internal processing diverges."

How to Extend This Document

When a new question comes up:

1. Add entry to the Table of Contents
2. Follow the structure: Quick definition → Details → Example → Why it matters
3. Keep plain-language definitions at the top of each section so readers who only want the gist don't have to read the full explanation
4. Cross-reference with specific sections of `CROSS_DOMAIN_CIRCUIT_PAPER.md` where relevant